

Week 11– Debugging and visualization tools

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.

Review from Last Week



- Integrated **LIDAR** and **motor systems** into the ROS 2 robot.
- Simulated a **2D LIDAR** in Gazebo and visualized scans in RViz.
- Connected a real **RPLIDAR A1** using the `rplidar_ros` driver.
- Wired and powered **DC motors**, **L298N driver**, **Arduino Nano**, and **encoders**.
- Flashed the **ROS-Arduino bridge** to control motor speed from the PC.
- Tested communication using **SerialTool** and verified encoder feedback.
- Robot now performs full sensor-actuator integration, ready for **motor calibration** next week.

Agenda for today's training session



Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
- Investigate robot behavior using simulation environments such as Gazebo for testing and debugging.

Problem we will be solving today:

- Vision in ROS 2 robot.
- ROS 2 Control Concepts & Simulation
- Navigation2 (Nav2) Framework
- Motion Planning with MoveIt 2
- Debugging and visualization tools

Vision Systems

- Cameras convert light into digital images (2D pixel grids).
- Each pixel stores color information (RGB or grayscale).
- In ROS, cameras are treated as sensors that publish `sensor_msgs/Image` messages.
- Debugging involves verifying both simulation and topic-level data flow.



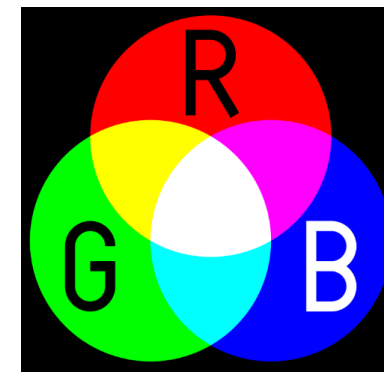
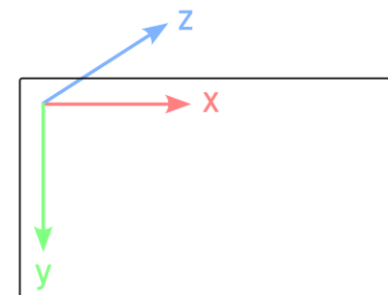
ROS 2 Camera Data Flow

Nodes → Topics → Messages → Visualization

- Camera driver node publishes `/camera/image_raw`.
- Camera info node publishes `/camera/camera_info`.
- Visualization tools (RViz, rqt_image_view) display the output.
- Debug tools (rqt_graph, ros2 topic list/echo, image_transport) verify connections.

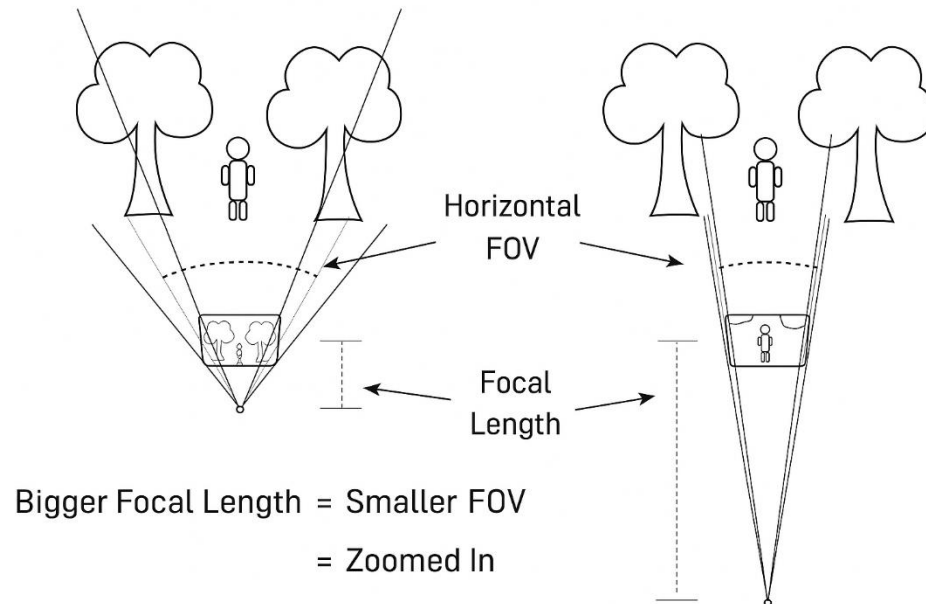
Camera Basics

- Converts light into a 2-D grid of pixels.
- Each pixel stores color values (RGB).
- ROS 2 treats cameras as **sensors** publishing `sensor_msgs/Image`.
- Typical topics:
 - `/camera/image_raw` (unprocessed image)
 - `/camera/camera_info` (metadata and calibration)



Focal Length and Field of View

- **Focal Length (f)**: distance between lens and image sensor.
- **Sensor Width (w)**: physical width of the camera's sensor.
- **Horizontal Field of View (FOV_h)** is the angular span the camera sees.



$$FOV_h = 2 \tan^{-1} \left(\frac{w}{2f} \right)$$

Focal Length and Field of View

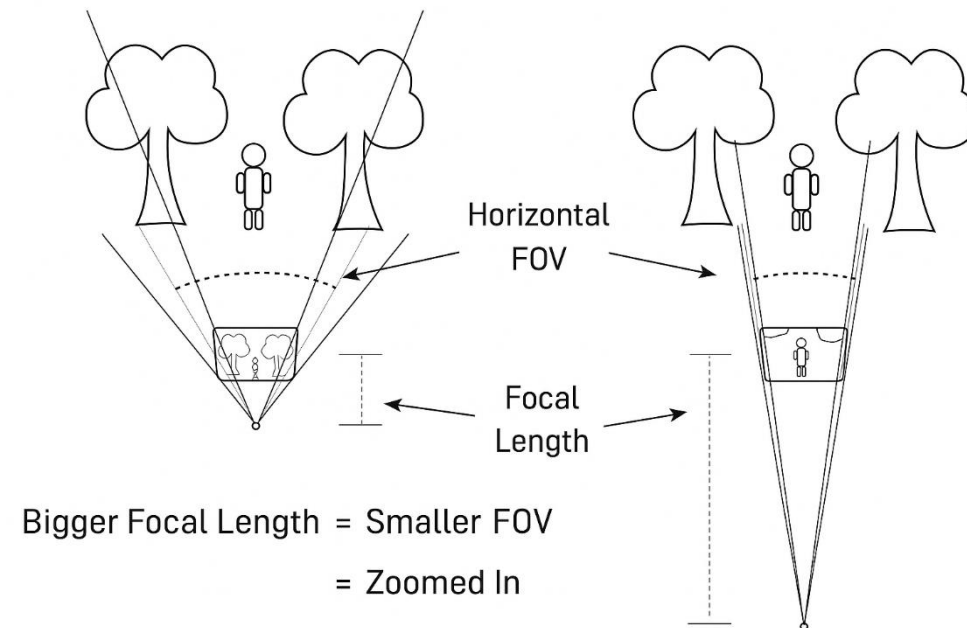
Example

Sensor width = 6 mm, $f = 4$ mm

$$FOV_h = 2 \tan^{-1} \left(\frac{6}{8} \right) \approx 73.7^\circ$$

Short focal length → wide view (zoom out)

Long focal length → narrow view (zoom in)



Camera Node and Topics

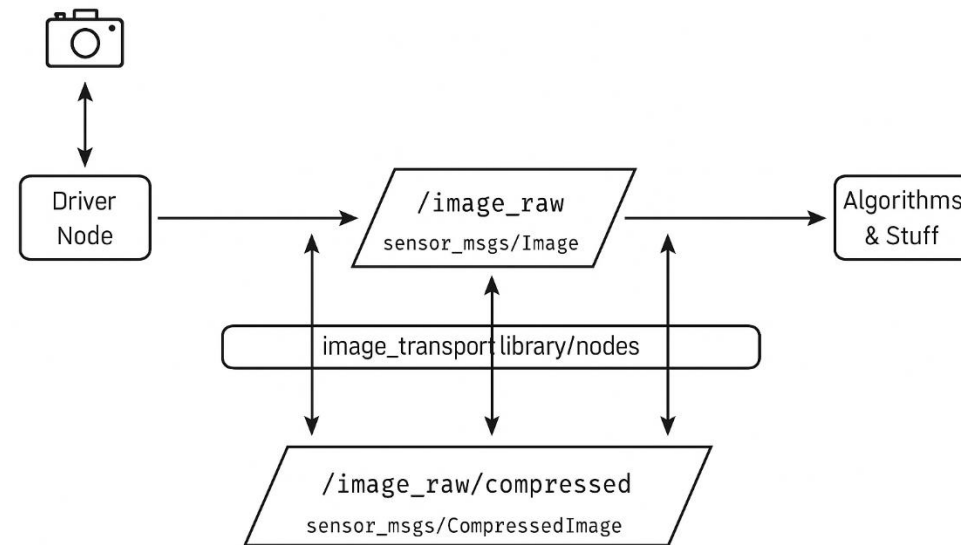
- **Driver node** communicates with camera hardware or simulation.
- Publishes `sensor_msgs/Image` messages.
- Metadata (resolution, distortion, etc.) sent via `sensor_msgs/CameraInfo`.
- ROS tools like `image_transport` manage compression and republishing.

Quick check:

```
bash
```

```
ros2 topic list
```

```
ros2 topic info /camera/image_raw
```



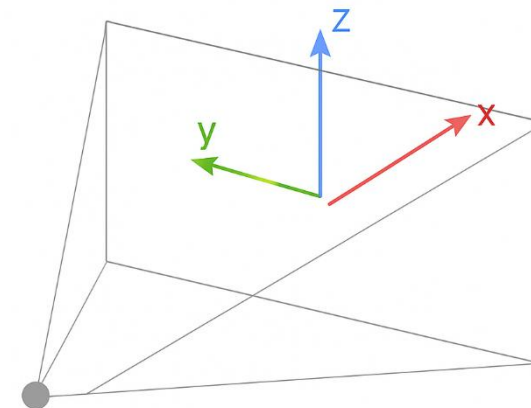
Step 1: Add Camera to URDF

Create `camera.xacro` in your description folder:

```
<joint name="camera_joint" type="fixed">
  <parent link="chassis"/>
  <child link="camera_link"/>
  <origin xyz="0.305 0 0.08" rpy="0 0 0"/>
</joint>

<link name="camera_link">
  <visual>
    <geometry>
      <box size="0.01 0.03 0.03"/>
    </geometry>
    <material name="red"/>
  </visual>
</link>
```

ROS Body Standard



camera_link

```
<gazebo reference="camera_link">
  <material>Gazebo/Red</material>
</gazebo>
```

Step 2: Add Optical Frame

Aligns image orientation with optical standards:

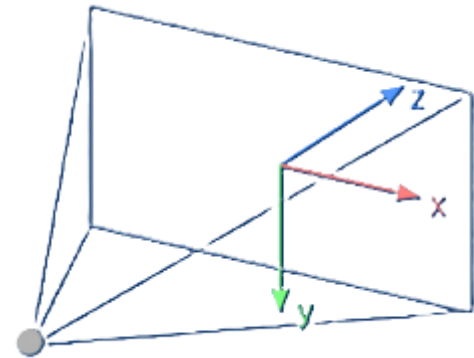
xml

```
<joint name="camera_optical_joint" type="fixed">  
  <origin xyz="0 0 0" rpy="{-pi/2} 0 {-pi/2}"/>  
  <parent link="camera_link"/>  
  <child link="camera_link_optical"/>  
</joint>  
  
<link name="camera_link_optical"/>
```

camera_link → ROS frame (x-forward, y-left, z-up)

camera_link_optical → optical frame (x-right, y-down, z-forward)

Image Standard



camera_link_optical

Step 3: Add Gazebo Camera Sensor

Insert this inside the `<gazebo reference="camera_link">` tag:

```
<sensor name="camera" type="camera">
  <update_rate>10</update_rate>
  <camera>
    <horizontal_fov>1.089</horizontal_fov>
    <image>
      <width>640</width>
      <height>480</height>
      <format>R8G8B8</format>
    </image>
    <clip><near>0.05</near><far>8.0</far></clip>
  </camera>
  <plugin name="camera_controller"
    filename="libgazebo_ros_camera.so">
    <frame_name>camera_link_optical</frame_name>
  </plugin>
</sensor>
```

Step 4: Rebuild and Launch

Rebuild your workspace and start simulation:

```
bash

cd ~/dev_ws
colcon build
source install/setup.bash
ros2 launch my_robot_bringup gazebo.launch.py
```

Check data flow:

```
bash

ros2 topic list
ros2 topic echo /camera/image_raw
```

Step 5: Visualize in RViz

1. Open RViz 2
2. Add **Image** display
3. Topic → `/camera/image_raw`
4. Fixed Frame → `base_link` or `camera_link`
5. If blank → set **Reliability Policy** to *Best Effort*

Tip: Use "Camera" display type to overlay live video on 3D world.

Key Debugging Tools

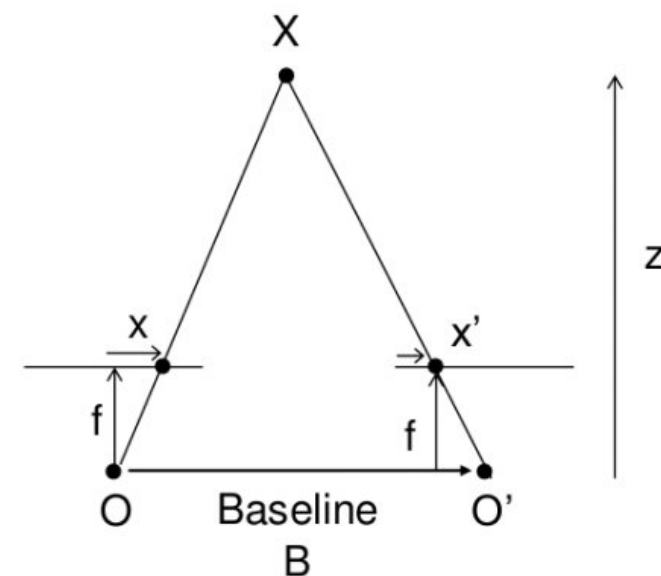
Tool	Purpose	Example Command
<code>ros2 node list</code>	Check if the driver node is running	<code>ros2 node list</code>
<code>ros2 topic list</code>	Confirm image topics exist	<code>ros2 topic list</code>
<code>ros2 topic info</code>	Inspect message types and publishers	<code>ros2 topic info</code> <code>/camera/image_raw</code>
<code>rqt_graph</code>	Visualize node/topic connections	<code>rqt_graph</code>
<code>rqt_image_view</code>	Display camera output	<code>rqt_image_view</code>
<code>image_transport</code>	Handle compression/decompression	<code>ros2 run image_transport</code> <code>republiish ...</code>

Depth Cameras: Seeing in 3D

- Depth cameras measure **distance per pixel**, enabling 3D perception.
- Used for **SLAM**, **object detection**, and **environment mapping**.
- Three main technologies:
 - **Structured Light** (IR pattern distortion – e.g., Kinect v1)
 - **Time-of-Flight** (light pulse delay – e.g., Kinect v2)
 - **Stereo Vision** (two cameras for disparity – e.g., OAK-D Lite)

$$\text{Depth} = \frac{f \cdot B}{d}$$

(where f = focal length, B = baseline, d = pixel disparity)



[OpenCV](#)

Depth Cameras in ROS 2

- Publish topics similar to RGB cameras:
`/camera/depth/image_raw` and `/camera/depth/camera_info`.
- Each pixel encodes depth (in `m` for float, or `mm` for int).
- Use `depth_image_proc` package for:
 - Depth $\rightarrow$ PointCloud2 conversion
 - RGB + Depth alignment (registration)
 - Depth normalization for visualization



Simulating a Depth Camera (Gazebo)

- Duplicate existing camera file:

```
camera.xacro → depth_camera.xacro
```

- Update sensor type:

```
xml
```

```
<sensor name="camera" type="depth">  
  <plugin>gazebo_ros_camera</plugin>  
  <update_rate>10</update_rate>  
</sensor>
```

- Add `min_depth` and `max_depth` parameters.
- Visualize in RViz using `/camera/depth/image_raw` or `/camera/points`.
- Depth image = grayscale map (black = near, white = far).

Using a Real Depth Camera

- Example: **OAK-D Lite** (Luxonis / DepthAI platform)
 - RGB + two stereo grayscale cameras.
 - Onboard **Intel Movidius VPU** for AI processing.
- Advantages:
 - Offloads computation from host.
 - Generates real-time depth maps and 3D point clouds.
- Connect via USB and verify topics:

```
bash
```

```
ros2 topic list | grep camera
```

ROS 2 Control Concepts & Simulation

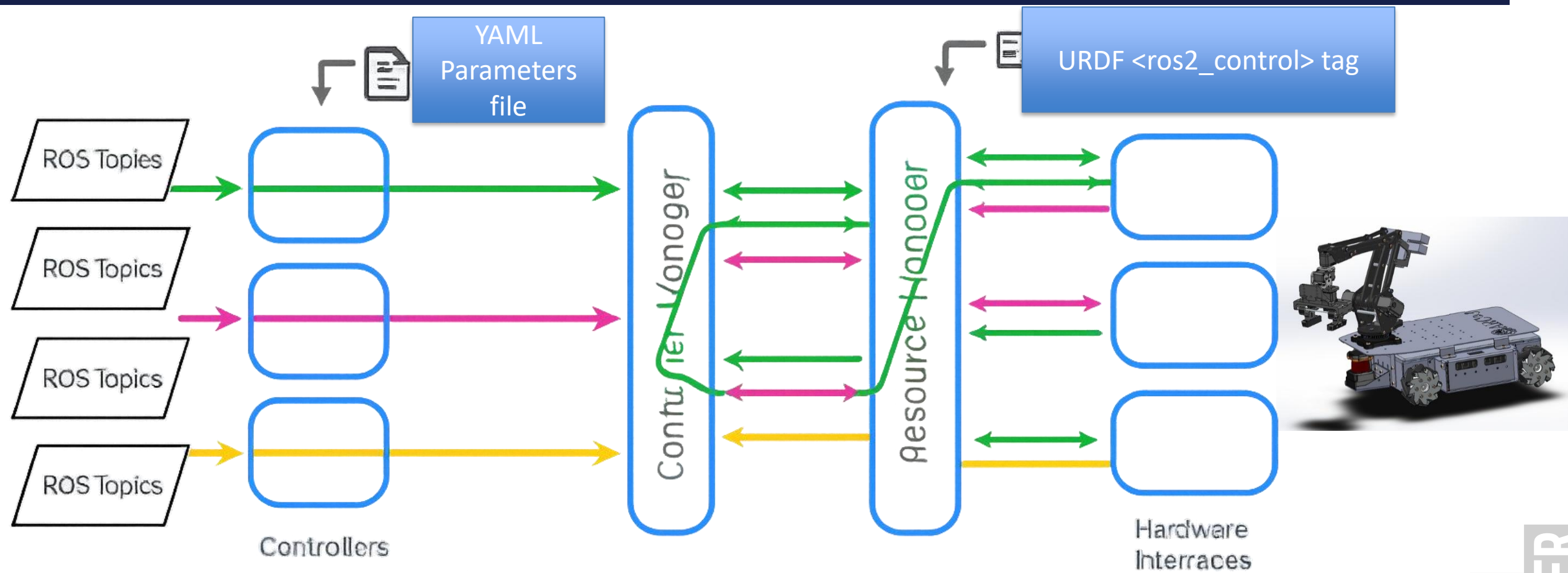
- **Every robot must read inputs → compute control → command actuators.**
- **Writing custom controllers for every hardware is inefficient.**
- **ros2_control provides a standard interface between:**
 - hardware drivers
 - control algorithms
 - the rest of ROS 2
- **Enables code reuse and hardware abstraction.**

Speed Requirement

Topics are good for data exchange, but controllers need real-time response.

- **ros2_control runs inside process memory (plugins, not nodes).**
- **Eliminates network delays.**
- **Suitable for motor loops running 100–1000 Hz.**

ros2_control Architecture



Controller Manager

- Central component that loads:
 - Hardware interface plugins
 - Controller plugins
- Manages their lifecycle: *configure* → *start* → *stop*.
- Uses a **resource manager** to track all command and state interfaces.

Hardware Interfaces

- Abstract layer between controllers and real/sim hardware.
- Two main types of interfaces:
 - **Command interfaces** → what we can control (velocity, position, torque).
 - **State interfaces** → what we can measure (position, velocity, current).

Example:

Joint	Command	State 1	State 2
left wheel	velocity	position	velocity
right wheel	velocity	position	velocity

Mathematical Model

For a two-wheel differential drive robot:

$$v = \frac{r}{2}(\omega_L + \omega_R), \quad \omega = \frac{r}{L}(\omega_R - \omega_L)$$

where

- r = wheel radius,
- L = wheel separation,
- ω_L, ω_R = wheel angular velocities.

The `diff_drive_controller` computes these automatically from `cmd_vel`.

Multiple Interfaces

A robot can have several hardware interfaces (e.g., arm + mobile base).

- Each adds its own command and state sets.
- The controller manager aggregates them via the **resource manager**.
- Defined in URDF under:

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
</ros2_control>
```

Controllers

- **Translate high-level ROS commands → actuator signals.**
- **Each controller claims certain resources (command interfaces).**
- **Common controllers:**
 - **diff_drive_controller**
 - **joint_state_broadcaster**
 - **effort_controllers / position_controllers**
- **Configured via YAML and loaded at runtime.**

YAML Configuration Example

```
controller_manager:  
  ros__parameters:  
    update_rate: 30  
    use_sim_time: true  
  
  diff_cont:  
    type: diff_drive_controller/DiffDriveController  
  
  joint_broad:  
    type: joint_state_broadcaster/JointStateBroadcaster
```

YAML Configuration Example – Cont.

```
diff_cont:  
  ros__parameters:  
    publish_rate: 30.0  
    base_frame_id: base_link  
    left_wheel_names: ['left_wheel_joint']  
    right_wheel_names: ['right_wheel_joint']  
    wheel_separation: 0.35  
    wheel_radius: 0.05
```

Gazebo Integration

```
<gazebo>  
  <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">  
    <parameters>$(find my_robot)/config/my_controllers.yaml</parameters>  
  </plugin>  
</gazebo>
```

- Plugin runs its own controller manager.
- Interfaces Gazebo physics with ros2_control.
- Uses YAML to auto-load controllers.

Running Controller Manager

```
bash
```

```
ros2 control list_hardware_interfaces  
ros2 run controller_manager spawner.py diff_cont  
ros2 run controller_manager spawner.py joint_broad
```

Then check:

```
bash
```

```
ros2 control list_controllers
```

Teleoperation

Remap teleop to new topic:

```
bash
```

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \  
--ros-args -r /cmd_vel:=/diff_cont/cmd_vel_unstamped
```

- Default topic for diff_drive_controller = `/controller_name/cmd_vel_unstamped`.
- If `use_stamped_vel = true`, expects `TwistStamped`.

Launch File Integration

```
python
```

```
diff_drive_spawner = Node(  
    package="controller_manager",  
    executable="spawner.py",  
    arguments=["diff_cont"],  
)  
joint_broad_spawner = Node(  
    package="controller_manager",  
    executable="spawner.py",  
    arguments=["joint_broad"],  
)
```

Then append them to the `LaunchDescription()` list.

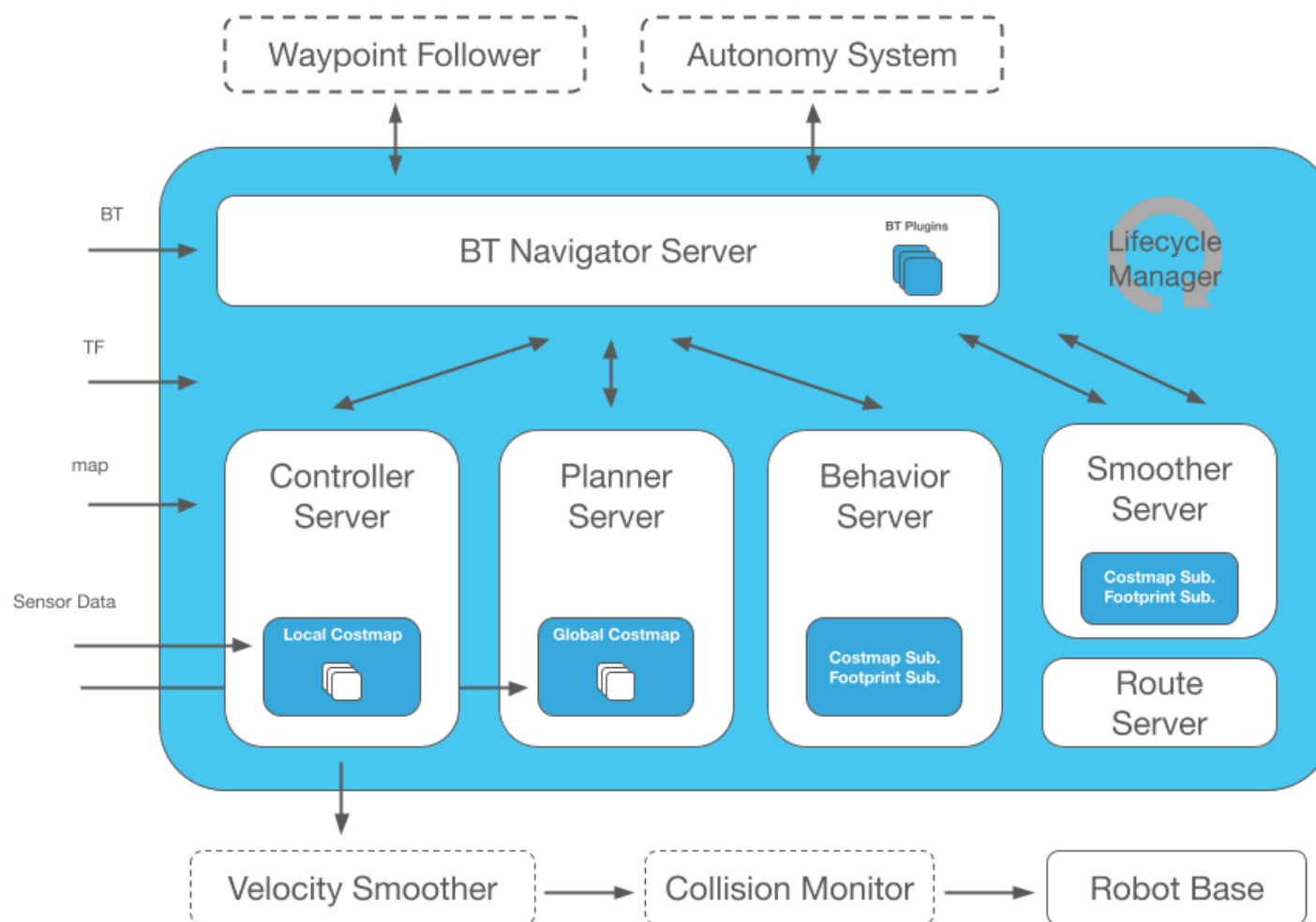
Key Debugging Commands

Purpose	Command
List interfaces	<code>ros2 control list_hardware_interfaces</code>
List controllers	<code>ros2 control list_controllers</code>
Start/stop controller	<code>ros2 control switch_controllers</code>
Check topics	<code>ros2 topic list</code>
Inspect frames	<code>ros2 run tf2_tools view_frames</code>

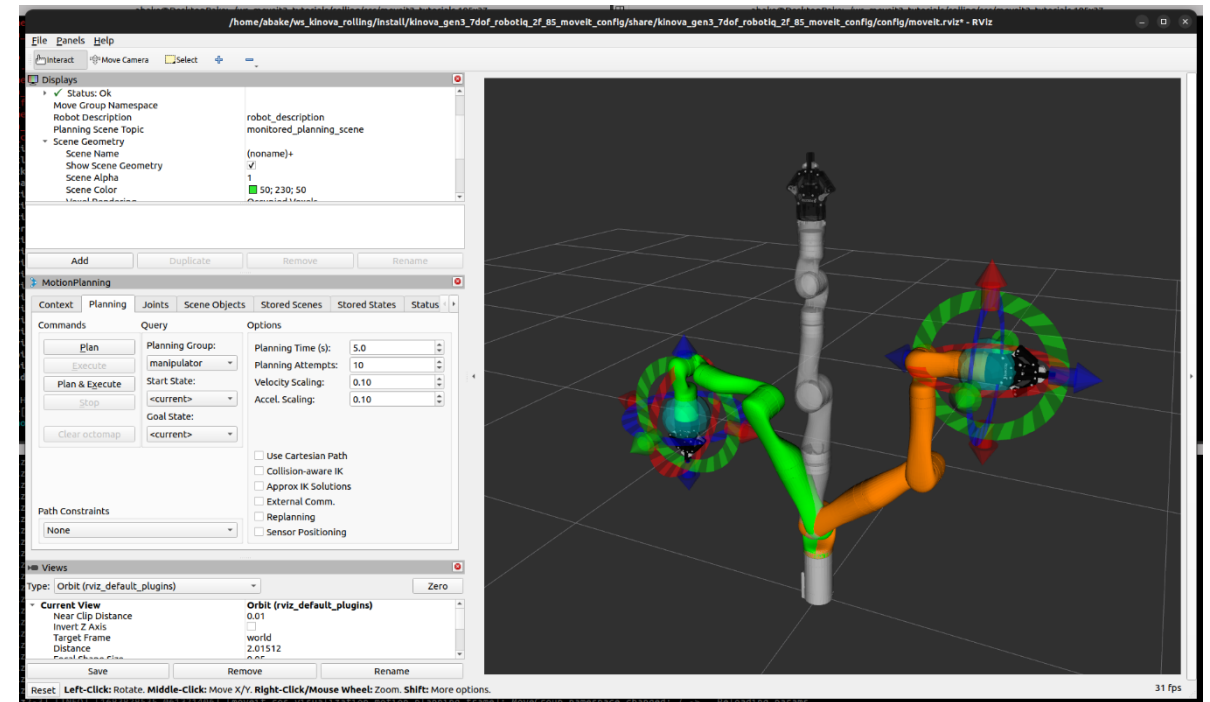
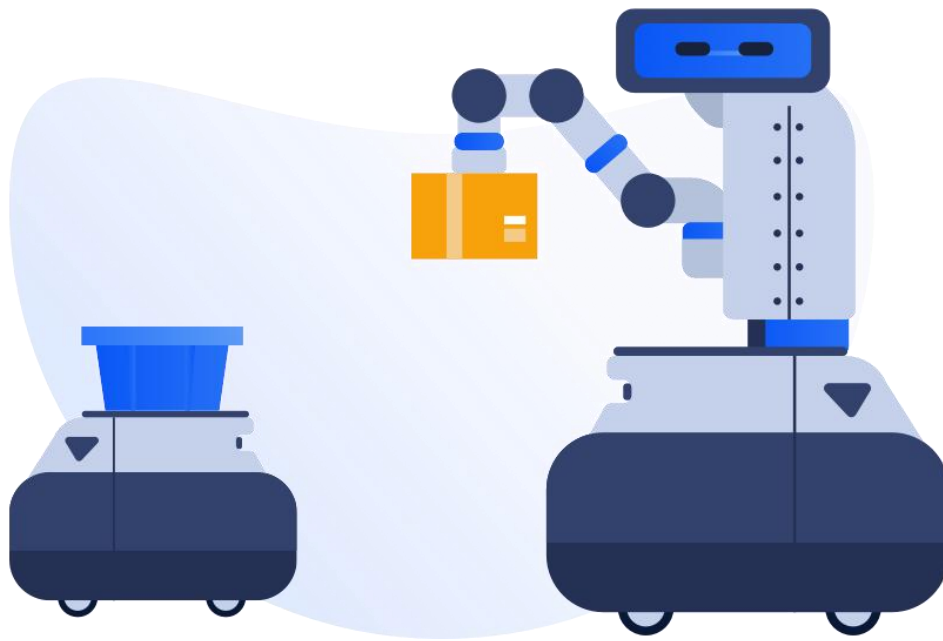
Navigation2 (Nav2) Framework

- **Successor to the ROS 1 Navigation Stack.**
- **Production-grade, modular navigation framework for ROS 2.**
- **Used by 100+ companies worldwide.**
- **Supports a wide range of robot types and tasks:**
 - **Differential drive, holonomic, ackermann, and legged bases.**
 - **Object following, waypoint missions, coverage, and more.**

Navigation2 Block Diagram



Motion Planning with MoveIt 2



<https://moveit.picknik.ai/main/index.html>

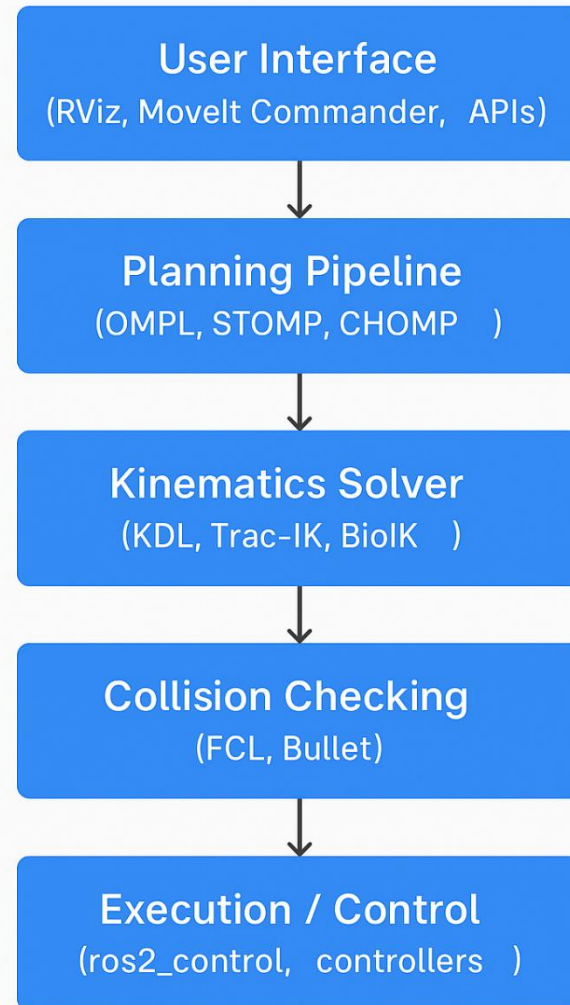
What is MoveIt 2?

- Open-source motion planning framework for ROS 2.
- Developed by **PickNik Robotics** and the ROS community.
- Provides a complete solution for robotic arms and manipulators:
 - **Planning** collision-free paths
 - **Kinematics** and **trajectory execution**
 - **Perception** and **grasping**
 - **Real-time control integration**

Key Capabilities

- **Motion Planning:** Generate feasible, collision-free paths.
- **Inverse Kinematics:** Solve for joint angles to reach a pose.
- **Collision Checking:** Real-time obstacle detection.
- **Path Optimization:** Smooth and shorten motion trajectories.
- **Perception Integration:** Fuse sensor data for environment awareness.
- **Control Interface:** Works with `ros2_control` and hardware drivers.

Movelt 2 Architecture



Core Components

Component	Description
Planning Scene	Maintains environment and robot model
Planning Pipeline	Generates motion plans using planners
Kinematics Solver	Computes joint solutions for end-effector
Trajectory Execution	Sends joint commands to controllers
MoveGroup Interface	Main API for planning and execution

Applications

- Industrial manipulators and cobots (UR, ABB, Fanuc).
- Service robots (fetching, grasping, placement).
- Research in motion optimization and grasp planning.
- Multi-arm and mobile-manipulator systems.

Summary



- **Perception:** Cameras & LiDAR provide data for visualization and sensing.
- **Control:** `ros2_control` connects controllers with robot hardware.
- **Navigation:** Nav2 handles mapping, planning, and obstacle avoidance.
- **Manipulation:** MoveIt 2 performs motion planning and execution.

THANK YOU.



WE ARE

HUMBER